

OmniAI Laboratory



AttriMem: Attribution-Guided Process Feedback for Agent Memory Learning

Qinfeng Li¹, Yuntai Bao¹, Xinyan Yu¹, Hongze Chen¹, Wenqi Zhang¹, Xuhong Zhang^{*1}

¹Zhejiang University

Effective memory is crucial for LLM agents, yet constructing it effectively remains challenging. A memory-construction policy decides what information to extract, store, update, compress, or discard as interactions accumulate. Heuristic memory methods rely on subjective, task-specific rules, which can misalign with downstream objectives and limit cross-task adaptability. RL-based methods, by contrast, learn from task feedback but mainly use outcome- or module-level rewards. These coarse signals indicate task success but cannot identify which intermediate memory contents support the final answer, creating a *fine-grained credit-assignment bottleneck*. However, constructing such process feedback is prohibitively difficult because intermediate memory decisions lack unique ground-truth targets, while the appropriate credit varies with the agent’s uncertain reasoning trajectory and therefore cannot be specified in advance. We propose **AttriMem**, an attribution-guided process-feedback framework for learning memory-construction policies with RL. AttriMem augments the global outcome reward with local rewards derived from token-level contributions to the final answer. Experiments on long-horizon dialogue question answering show that AttriMem outperforms retrieval-based, heuristic, and RL-based baselines, generalizes across benchmarks and answer models, stabilizes RL optimization.

1 Introduction

As LLM agents are used in tasks with long interactions and large amounts of information, they need to remember and use past interactions to make good decisions (Park et al., 2023; Zhong et al., 2024; Wu et al., 2024). This is challenging because the information needed to answer questions is often scattered across sessions and mixed with irrelevant details (Maharana et al., 2024; Liu et al., 2024). To handle such long interaction histories, a memory-construction policy incrementally builds an external memory store by deciding what information to extract, store, update, merge, compress, or discard. Therefore, **a key problem is how to learn such a memory-construction policy (i.e., memory learning) to preserve task-relevant information while filtering out unnecessary context.**

However, as shown in Table 1, existing approaches to constructing such memory remain limited in several important aspects. For example, **heuristic memory methods** (Yan et al., 2025; Xu et al., 2026a; Fang et al., 2025; Chhikara et al., 2025; Lewis et al., 2020) rely on manually designed rules to determine what information should be stored, retained, or retrieved, making memory construction highly subjective. On one hand, such subjectivity can easily lead to objective mismatch (Zhang et al., 2026): information that appears salient under the designer’s intuition may not actually matter for downstream decisions. On the other hand, these methods are often tightly coupled to a particular task setting, which limits their generalizability (Zhou et al., 2026). When the task objective changes, the original memory policy often fails to adapt flexibly, requiring further manual adjustment.

RL-based memory-learning methods (Yue et al., 2026; Shen et al., 2026; Wang et al., 2025), by contrast, learn memory behavior from task-grounded feedback, enabling optimization of memory policies for diverse tasks.

Paradigm (exemplar)	Task-Learned	Process Feedback	Additional Reward Signal	Token-Level Feedback
Heuristic Memory (Zhong et al., 2024)	✗	-	-	-
Task-Performance RL (Wang et al., 2025)	✓	✗	✗	✗
Outcome-Redistribute Rewards (Yue et al., 2026)	✓	✓	✗	✗
QA-Derived Action Rewards (Shen et al., 2026)	✓	✓	✓	✗
AttriMem (ours)	✓	✓	✓	✓

Table 1 Comparison of memory-learning paradigms. AttriMem is the only paradigm that learns from task feedback, introduces feedback beyond outcomes, and provides operation- and token-level guidance for memory learning.

However, existing methods still largely rely on final task performance for supervision, leaving intermediate memory actions with sparse rewards and creating a fine-grained credit-assignment bottleneck that hinders effective policy learning: such signals only indicate whether the task succeeds but provide little guidance on which memory actions are responsible for success or failure. A representative paradigm is **task-performance RL**, which optimizes memory construction using downstream performance signals, such as answer correctness (Wang et al., 2025). Although such rewards make memory learning task-grounded, they offer little direct guidance for intermediate memory actions, because an answer-only signal cannot specify how each memory action contributed to that outcome. To provide feedback for intermediate processes, **outcome-redistribute rewards methods**, such as Mem-T (Yue et al., 2026), redistribute final feedback to intermediate nodes through sampled memory-action trees, thereby propagating the outcome rewards of leaf trajectories to their ancestor memory-action nodes. However, this feedback fundamentally remains a redistribution of the final feedback and does not introduce any additional information, leaving the feedback still sparse.

More advanced **QA-derived action-reward methods**, such as MemBuilder (Shen et al., 2026), synthesize auxiliary QA pairs and reward memory actions according to whether their contents are retrieved and help answer these questions correctly, thereby providing richer reward signals for memory learning. However, these rewards are still aggregated from final QA performance, rather than derived from direct evidence of how each intermediate memory action contributes to the answer, making the supervision insufficiently fine-grained. In particular, an action-level signal evaluates a memory action only as a whole and therefore provides no targeted guidance for improving its internal content, even though its utility may depend on only a few critical tokens or spans, such as an entity, value, or timestamp. This motivates finer-grained feedback that can evaluate and improve the specific content produced by each action, down to individual tokens.

These limitations expose a fundamental challenge in existing RL-based memory learning. Memory operations ideally require direct and fine-grained feedback, yet such feedback is intrinsically difficult to obtain because memory construction is itself a long-horizon problem over long contexts. The **central difficulty** is to determine which pieces of evidence actually support the ongoing reasoning trajectory and the final answer. First, existing memory benchmarks rarely provide ground-truth process signals for intermediate memory decisions. More fundamentally, **constructing such ground-truth is prohibitively difficult**, because intermediate memory decisions do not have a unique correct target: which memory evidence should receive credit depends on the reasoning trajectory taken by the agent, and this trajectory is highly uncertain. Consequently, **no universal and static process ground-truth can faithfully supervise intermediate memory**.

This motivates us to seek an alternative source of process supervision: can task performance be converted into dense, fine-grained token-level credit signals for memory-process outputs? **Our key insight is to use the final answer to find the useful tokens in the intermediate memory process and convert their contributions into process rewards**.

Building on this insight, we propose AttriMem, an attribution-guided process-feedback framework for RL-based memory learning. During training, AttriMem follows the standard memory learning pipeline: the policy constructs intermediate memory outputs, a fixed retrieval-and-answering interface produces the final answer, and an outcome reward evaluates overall task performance. Beyond this global reward, AttriMem quantifies how much each token in the intermediate memory outputs contributes to the final answer. Concretely, it masks different subsets of memory tokens and uses the resulting changes in the final-answer score to estimate each token’s contribution. These token-level scores are then mapped back to the memory process that produced them and converted into local process rewards. As a result, the policy receives feedback not only on overall task success, but also on the specific content within each memory action.

The **contributions** of our work are summarized as follows.

1. We identify a fine-grained credit-assignment bottleneck in memory learning: existing RL-based methods provide only outcome- or action-level feedback, leaving the utility of specific memory contents unspecified.
2. We analyze why fine-grained process supervision is difficult to obtain: memory benchmarks usually lack ground-truth process signals for intermediate memory, and such ground-truth is prohibitively difficult to construct.
3. We propose AttriMem, an attribution-guided process-feedback framework that treats the final answer as a credit-assignment target and traces its supporting evidence back to intermediate memory process reward.
4. Extensive experiments show that token-level attribution enables more effective memory-policy optimization than outcome-only or action-level rewards, allowing AttriMem to achieve stronger task performance and more reliable memory-construction behavior.

2 Related Work

Training-free and heuristic memory. Retrieval-augmented methods query external memory at inference time (Lewis et al., 2020), and heuristic memory methods use hand-crafted rules or scoring functions for writing, retaining, or compressing (Yan et al., 2025; Xu et al., 2026a; Fang et al., 2025; Chhikara et al., 2025). However, their memory policies are predefined, which can lead to an objective mismatch.

Outcome-derived RL for memory learning. RL-based methods learn memory behavior from task rewards, making memory learning more task-grounded. Methods such as Mem- α (Wang et al., 2025) optimize memory policies using task-performance-derived feedback, while Mem-T (Yue et al., 2026) redistributes final rewards to intermediate nodes in sampled memory-operation trees. However, these signals remain outcome-derived: they indicate downstream success but provide limited evidence about which memory operation, content span, or token supports the final answer.

QA-aggregated RL methods. The closest work to ours is MemBuilder (Shen et al., 2026), which uses synthetic session-level QA to evaluate the memory bank and forms action-level rewards from QA performance and retrieval usage. Although this provides denser supervision than final task success alone, the reward is still too coarse to identify which operation, memory entry, or content span should receive credit. In contrast, AttriMem derives token-level process feedback from answer contribution and aligns it with the specific content produced within each intermediate memory operation.

3 Preliminaries

In this paper, we address the task of long-term dialogue question answering. Given a conversation history consisting of T sessions $\mathcal{H} = \{h_t\}_{t=1}^T$, where each session h_t comprises multi-turn utterances at timestamp t , and a question q posed after timestamp T , the goal is to generate an accurate answer based on information distributed across the entire history. The agent operates in two phases.

- **Memory construction.** An external memory bank $\mathcal{M}$ compresses and organizes historical information for selective retrieval. At each session t , the agent updates $\mathcal{M}_t$ by choosing memory operations a_t^{mem} based on session h_t and previous memory $\mathcal{M}_{t-1}$. We denote by $\Delta\mathcal{M}_t$ the textual memory content produced or modified by this update.
- **Question answering with memory retrieval.** To answer question q , a fixed retriever retrieves from $\mathcal{M}_T$ and an answer model (e.g., Claude) generates an answer $\hat{y}$.

In this work, **our goal** is to improve question answering performance by improving the quality of memory construction; retrieval and answering provide evaluation and reward but are not optimized actions. The optimized rollout is the memory-construction trajectory

$$\tau^{\text{mem}} := \{(h_t, \mathcal{M}_{t-1}, a_t^{\text{mem}}, \Delta\mathcal{M}_t)\}_{t=1}^T. \quad (1)$$

The **core challenge** is that the reward for each intermediate memory action is delayed (it arrives after fixed

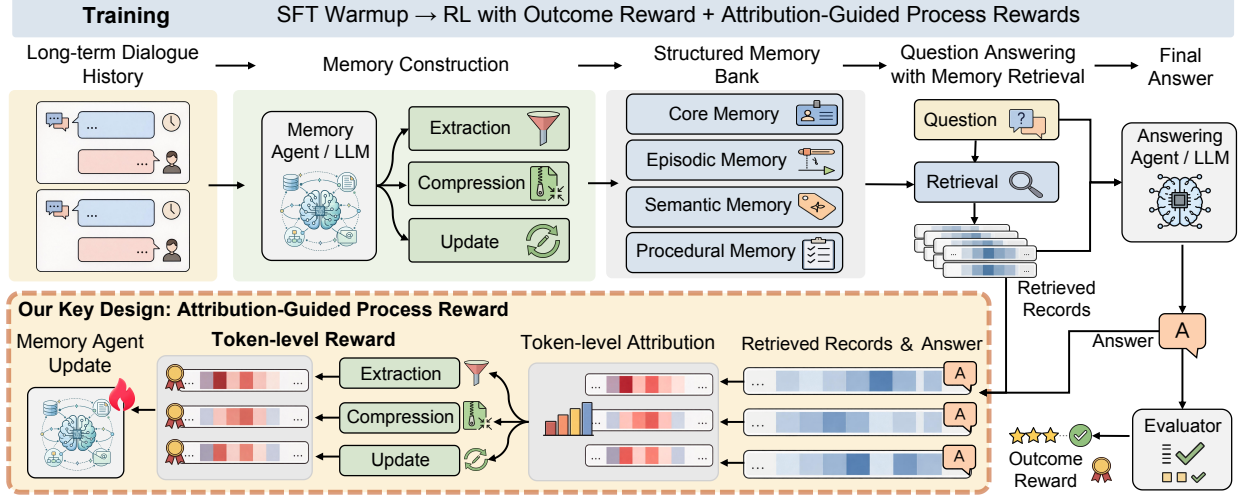


Figure 1 Overview of AttriMem. AttriMem constructs a structured memory bank from long-term dialogue through extraction, compression, and update, then answers questions via memory retrieval. During RL training, token-level attribution produce local process rewards, which complement the global outcome reward for fine-grained memory learning.

retrieval and answering), and final task success/failure provides no direct signal about which memory operation was beneficial or detrimental.

3.1 Memory Architecture

Here we provide a general description of the generic memory architecture for question answering in the context of multi-turn, long-horizon conversations, since the designs of recent memory agents are homogeneous at the high-level (Shen et al., 2026; Yue et al., 2026; Zhou et al., 2025).

Following the design of Shen et al. (2026), our memory architecture consists of four specialized memory types:

- $\mathcal{M}^{\text{core}}$: Core memory, fixed-size user profile (identity, preferences, relationships);
- $\mathcal{M}^{\text{epi}}$: Episodic memory, time-stamped event records with temporal chaining;
- $\mathcal{M}^{\text{sem}}$: Semantic memory, factual knowledge about user-specific entities;
- $\mathcal{M}^{\text{proc}}$: Procedural memory, step-by-step processes and workflows.

Each component $m \in \{\text{core}, \text{epi}, \text{sem}, \text{proc}\}$ has its own action space $\mathcal{A}^{(m)}$ (e.g., ADD, UPDATE, MERGE) and is processed by the same LLM with type-specific prompts.

3.2 Training a Memory Agent

Memory agent training with RL. We follow the general problem setup of MEM1 (Zhou et al., 2025) and consider an interactive, memory-augmented agent. In our setting, the learned policy is the memory construction policy $\pi_{\theta}(a_t^{\text{mem}} | h_t, \mathcal{M}_{t-1})$, whose likelihood is defined over memory operations rather than retrieval or answer-generation actions. After the memory trajectory τ^{mem} is constructed, the retriever-reader produces $\hat{y}$ and the final answer quality supplies the outcome reward. The learning problem can therefore be written as KL-constrained reward maximization over memory-construction rollouts:

$$\max_{\theta} \mathcal{J}(\theta) = \max_{\theta} \mathbb{E}_{\substack{(q, \mathcal{H}) \sim \mathcal{D}, \\ \tau^{\text{mem}} \sim \pi_{\theta}(\cdot | \mathcal{H})}} \left[R(q, \tau^{\text{mem}}) - \beta \text{KL}(\pi_{\theta} \| \pi_{\text{ref}}) \right], \quad (2)$$

where $\mathcal{D}$ is the training distribution over histories and questions, $R(q, \tau^{\text{mem}})$ is the measurement for rollout quality and often includes outcome and process rewards, π_{ref} is the reference model, and β is a constant.

Memory agent training with GRPO. GRPO is commonly used for RL training for its efficiency since it does not require a value model (Shao et al., 2024). The *outcome-only* variant of GRPO is often used for *RL with verifiable reward (RLVR)*. For each training example, the policy samples G memory-construction trajectories. A frozen retriever then retrieves information from each resulting memory store, and a separate answer model (e.g., Claude) generates the final answer; neither component is part of the trained policy. The resulting answer rewards are normalized within the group:

$$\max_{\theta} \mathcal{J}(\theta) = \max_{\theta} \mathbb{E}_{(q, \mathcal{H}) \sim \mathcal{D}, \{\tau_i^{\text{mem}}\}_{i=1}^G \sim \pi_{\text{old}}(\cdot | \mathcal{H})} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T} \sum_{t=1}^T \min \left(\rho_{i,t} \hat{A}_i^{\text{out}}, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i^{\text{out}} \right) - \beta \text{KL}(\pi_{\theta} \| \pi_{\text{ref}}) \right],$$

$$\text{with } \rho_{i,t} := \frac{\pi_{\theta}(a_{i,t}^{\text{mem}} | h_{i,t}, \mathcal{M}_{i,t-1})}{\pi_{\text{old}}(a_{i,t}^{\text{mem}} | h_{i,t}, \mathcal{M}_{i,t-1})}, \hat{A}_i^{\text{out}} = \frac{R_i^{\text{out}} - \text{mean}(\{R_j^{\text{out}}\}_{j=1}^G)}{\text{std}(\{R_j^{\text{out}}\}_{j=1}^G)},$$
(3)

where G is group size and π_{old} is the old policy model. Under this formulation, GRPO with only outcome reward applies a single feedback to account for the quality of the entire trajectory. This reward sparsity often results in training inefficiency and could fail to penalize incorrect intermediate actions as long as the outcome is correct.

Context attribution. We formulate context attribution following Cohen-Wang et al. (2024). Given a context c and response $y \in \mathcal{Y}$, we partition the context into a set of sources for attribution: $\mathcal{Z} = \{z_i\}_{i=1}^{|\mathcal{Z}|}$. An attribution method is then defined as $\phi : \mathcal{Z} \times \mathcal{Y} \rightarrow \mathbb{R}^{|\mathcal{Z}|}$, where $\phi(z_i, y; c)$ quantifies the importance (or contribution) of source z_i on response y under full context c . Conceptually, the attribution score can be expressed as the change in response score when a source is removed:

$$\phi(z_i, y; c) \approx F(y | c) - F(y | c \setminus z_i),$$
(4)

where $F(y | c)$ denotes a measurement (e.g., model output logits) of response y under context c .

4 Method

4.1 Problem Formulation

We study long-term dialogue question answering with an explicit memory construction stage. Given a dialogue history $\mathcal{H} = \{h_t\}_{t=1}^T$ and a question q asked after the history, the system first constructs a memory store $\mathcal{M}_T$ from $\mathcal{H}$ and then answers q using a fixed retrieval-and-reading interface over $\mathcal{M}_T$. In this paper, AttriMem focuses on learning the *memory construction policy*: What information should be extracted, updated, merged, compressed, and retained in the memory store? Thus, performance improvements should be attributed to better memory construction; the learned policy produces memory records that are more complete, less noisy, and more useful for later retrieval, reasoning, and question answering. The retriever and answer generator are used for question answering, but they are not the target of our method.

Formally, at each session t , a memory construction policy π_{θ} observes the current session h_t and the previous memory store $\mathcal{M}_{t-1}$, and produces a memory-construction action a_t^{mem} . The action updates the memory store as

$$\mathcal{M}_t = U(\mathcal{M}_{t-1}, h_t, a_t^{\text{mem}}),$$
(5)

where $U(\cdot)$ denotes the memory update operator. Let $\Delta \mathcal{M}_t$ denote the textual memory content produced or modified by this transition, yielding the memory-construction trajectory $\tau^{\text{mem}} = \{(h_t, \mathcal{M}_{t-1}, a_t^{\text{mem}}, \Delta \mathcal{M}_t)\}_{t=1}^T$. After processing all sessions, a fixed retriever-reader interface produces an answer

$$\hat{y} = G(q, \text{Ret}(q, \mathcal{M}_T)),$$
(6)

where $G(\cdot)$ denotes the answer generator. The training signal is derived from the final answer quality, but optimized actions are the memory construction actions $\{a_t^{\text{mem}}\}_{t=1}^T$. This formulation isolates the central question of the paper: *How can we train a model to build a better memory store?*

Why attribution is needed

Outcome can tell whether the final answer is correct, but it cannot tell which memory operation caused the success or failure. This is especially problematic for long dialogue histories, where many memory records are irrelevant, redundant, or only partially useful. **Attribution-guided reward adds information that is absent from the final score:** it identifies which constructed memory content actually supports the answer. Based on this, AttriMem changes the credit-assignment signal for memory construction.

4.2 Attribution-based Process Reward

To resolve reward impoverishment, we assign process rewards to intermediate memory actions sampled from the agent.

Decomposed policy gradient. Our design of process rewards optimizes the memory construction policy. Based on the GRPO formulation, the policy gradient over memory-construction actions is as follows (ignoring the KL constraint term for simplicity):

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\substack{(q, \mathcal{H}) \sim \mathcal{D}, \\ \{\tau_i^{\text{mem}}\}_{i=1}^G \sim \pi_{\text{old}}(\cdot | \mathcal{H})}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T} \sum_{t=1}^T \rho_{i,t} \hat{A}_{i,t} \nabla_{\theta} \log \pi_{\theta}(a_{i,t}^{\text{mem}} | h_{i,t}, \mathcal{M}_{i,t-1}) \right], \quad (7)$$

$$\text{where } \rho_{i,t} := \frac{\pi_{\theta}(a_{i,t}^{\text{mem}} | h_{i,t}, \mathcal{M}_{i,t-1})}{\pi_{\text{old}}(a_{i,t}^{\text{mem}} | h_{i,t}, \mathcal{M}_{i,t-1})}, \hat{A}_{i,t} := \hat{A}_i^{\text{out}} + \lambda \hat{A}_{i,t}^{\text{proc}}.$$

where $\hat{A}_i^{\text{out}}$ is the within-group outcome advantage, $\hat{A}_{i,t}^{\text{proc}}$ is the within-group process advantage and is derived from r_t according to GRPO advantage estimation, and λ controls the weight of the process signal. The outcome advantage broadcasts final-answer quality to all memory actions in the trajectory, as in outcome-only GRPO, while the process advantage gives local attribution-based credit to $\Delta \mathcal{M}_{i,t}$.

Measuring the contribution of an action on the final answer. The **intuition** is to estimate the counterfactual contribution of intermediate steps: *if a source (e.g., token, series of tokens) were not present in the trajectory, how much would the final outcome change?*

Let $\Delta \mathcal{M}_t$ denote the textual memory content produced or modified by a_t^{mem} when updating $\mathcal{M}_{t-1}$ to $\mathcal{M}_t$. To estimate the contribution of the t -th memory action, we measure the contribution of $\Delta \mathcal{M}_t$ to the generated answer $\hat{y}$. Since $\Delta \mathcal{M}_t$ is a direct consequence of action a_t^{mem} , attributing the contribution of $\Delta \mathcal{M}_t$ to the final answer provides a proxy for the quality of a_t^{mem} . We treat $\Delta \mathcal{M}_t$ as a collection of textual sources (e.g., memory records), which are further decomposed into tokens for attribution. Specifically, we compute $\phi((\Delta \mathcal{M}_t)^{(i)}, \hat{y}; c)$ for the i -th token of $\Delta \mathcal{M}_t$, where ϕ is an attribution method, $\hat{y}$ is the generated answer, and c is the attribution context.

In this paper, we adopt *ContextCite* (Cohen-Wang et al., 2024), which fits a lightweight linear attribution model using randomly ablated context variants. Specifically, it approximates the response score as a linear function of whether each context source is retained. Although ContextCite evaluates 32 ablated variants, they can be processed in parallel, limiting the additional wall-clock latency to approximately one batched forward pass. We defer further implementation details to Appendix B.

Computing attribution rewards. We then derive process rewards based on the attribution results. Given $\Delta \mathcal{M}_t$, we compute the reward for a_t^{mem} via $r_t = \frac{1}{\sqrt{t}} \sum_{j=1}^{l_t} \phi((\Delta \mathcal{M}_t)^{(j)}, \hat{y}; c)$, where l_t is the token length of $\Delta \mathcal{M}_t$. The normalization is designed to mitigate length bias towards longer memory updates.

Remark (Credit-assignment perspective). *It is important to distinguish AttriMem from methods that merely add an auxiliary reward. The process reward in AttriMem is a credit-assignment mechanism: it provides information about the quality of intermediate decisions that is fundamentally unavailable in any outcome-based signal, regardless of how that signal is redistributed over time. By targeting each memory module with a reward derived from the causal contribution of its selected content to the final answer, AttriMem makes it possible to locally optimize individual memory components rather than relying on end-to-end RL to shape all modules jointly through a single scalar.*

Method	LoCoMo	LongMemEval	PerLTQA
<i>Training-Free Retrieval</i>			
RAG-Session [†]	70.35	66.75	79.21
RAG-Utterance [†]	74.87	69.00	77.23
<i>Heuristic Memory Construction</i>			
Mem0 [†]	51.64	47.00	62.04
MIRIX [†]	77.48	73.25	83.11
MemoryOS	52.92		64.08
A-Mem	68.98	60.00	74.66
LightMem	80.53	63.50	80.30
GAM	79.50	73.25	81.59
<i>RL-based Memory Learning</i>			
Memory-R1 [†]	62.74	–	–
Mem-T	75.67		76.58
MemBuilder (Qwen3-4B)	69.74	56.00	78.33
+ SFT	78.70	80.50	82.36
+ RL	69.84	58.00	79.41
+ SFT + RL	80.01	81.75	83.74
Ours (Qwen3-4B)	67.88	55.75	78.32
+ SFT	78.40	80.25	81.52
+ RL _{Tok}	72.05	61.25	80.58
+ SFT + RL _{Tok}	82.48	83.25	84.49

Table 2 Overall performance comparison. Results marked with [†] are reported in the original MemBuilder paper.

5 Experiments

5.1 Experimental Setup

Datasets and metric. We use LongMemEval (Wu et al., 2024) as the training dataset, and evaluate AttriMem on long-horizon dialogue-based question answering using another two benchmarks: LoCoMo (Maharana et al., 2024) and PerLTQA (Du et al., 2024). The two benchmarks are used for out-of-domain evaluation under a zero-shot transfer setting, enabling assessment of both in-domain memory policy learning and cross-domain generalization. We report accuracy as the primary evaluation metric across all benchmarks.

Baselines. We compare AttriMem against three categories of baselines. Training-free retrieval methods include RAG-Session and RAG-Utterance (Lewis et al., 2020), which directly retrieve historical context at the session or utterance level. Heuristic-based memory methods include Mem0 (Chhikara et al., 2025), MIRIX (Wang and Chen, 2025), MemoryOS (Kang et al., 2025), A-Mem (Xu et al., 2026b), and GAM (Wu et al., 2026), which construct memory using predefined prompts without task-driven policy optimization. RL-based memory methods include Memory-R1 (Yan et al., 2026), Mem-T (Yue et al., 2026), and MemBuilder (Shen et al., 2026), which optimize memory-construction policies using downstream task feedback or intermediate rewards. For MemBuilder and AttriMem, we additionally report the base, +SFT, +RL, and +SFT+RL variants to disentangle the effects of supervised initialization and RL optimization.

Implementation details. Since MemBuilder (Shen et al., 2026) is the most closely related recent work, we follow its backbone and memory architecture to enable a controlled comparison. Specifically, both MemBuilder and AttriMem use Qwen3-4B and undergo SFT warmup followed by GRPO-based RL (Shao et al., 2024). We keep the SFT initialization, optimization budget, shared hyperparameters, retrieval-and-answering interface, decoding configuration, and evaluation protocol consistent. We use 3,000 SFT steps and 400 RL steps, with detailed hyperparameters provided in Appendix B.

5.2 Main Results

Overall performance. We compare AttriMem with different baselines. As shown in Table 2, **the fine-grained attribution-based reward in AttriMem is consistently effective**. Despite comparable SFT performance, after RL, AttriMem reaches 82.48, 83.25, and 84.49, outperforming MemBuilder by 2.47, 1.50, and 0.75 points on the three benchmarks, respectively. **AttriMem achieves the best performance across all three benchmarks**. It surpasses the strongest prior methods by 1.95 points on LoCoMo, 1.50 points on LongMemEval, and 0.75

Method	LoCoMo	LongMemEval	PerLTQA
Ours (Qwen3-4B)	67.88	55.75	78.32
+ RL_{Out}	70.19	58.25	79.61
+ RL_{Act}	71.10	59.75	79.98
+ RL_{Tok}	72.05	61.25	80.58
+ SFT + RL_{Out}	79.46	81.50	83.69
+ SFT + RL_{Act}	81.32	82.00	84.05
+ SFT + RL_{Tok}	82.48	83.25	84.49

Table 4 Ablation study on reward granularity.

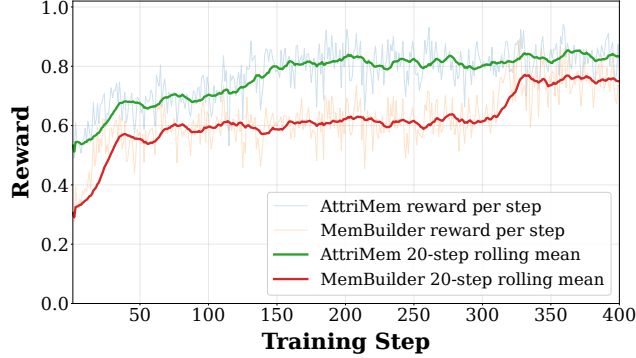


Figure 3 Reward evolution during RL training.

Complementarity between SFT and RL. We examine whether attribution-guided RL improves memory construction beyond SFT initialization. As shown in Table 2, **attribution-guided RL consistently improves performance with or without SFT**. Across LoCoMo, LongMemEval, and PerLTQA, RL_{Tok} improves the base model by 2.26–5.50 points and the SFT model by 2.97–4.08 points. Moreover, $SFT+RL_{Tok}$ achieves the best results, confirming that RL further optimizes memory construction beyond supervised imitation.

5.4 Further analysis

Intermediate memory quality. We use a GPT-based judge to compare intermediate memory, with details provided in Appendix B. As shown in Table 5, **token-level attribution improves the intermediate memory process itself**. Across LoCoMo, LongMemEval, and PerLTQA, Ours_{Tok} achieves win rates of 73.91%, 72.94%, and 77.78% over MemBuilder, respectively. It is also consistently preferred over Ours_{Out} and Ours_{Act}, with win rates of 61.11–71.43% and 58.97–61.80%.

RL training dynamics. We track reward evolution during RL training. As shown in Figure 3, **token-level attribution provides a more stable and steadily improving optimization signal than aggregated rewards**. AttriMem rises consistently, reaches a high-reward region midway through training, and remains stable. In contrast, MemBuilder plateaus after early gains and improves only through a sharp surge around step 320, indicating delayed and less stable optimization.

Inference token efficiency. We report average end-to-end tokens per question, including amortized memory construction and question answering. As shown in Figure 4, **AttriMem achieves the best performance–efficiency trade-off and lies on the Pareto frontier**. It attains the highest accuracy with token consumption comparable to MemBuilder and substantially lower than Mem-T and GAM. Although A-Mem uses fewer tokens, its accuracy is considerably lower.

6 Discussion and Future Work

Computational overhead. Estimating token contributions requires evaluating randomly masked variants, introducing additional computation during training. However, this overhead is negligible. Since the variants

Comparison	LoCoMo	LongMemEval	PerLTQA
Ours_{Tok} vs. MemBuilder	73.91	72.94	77.78
Ours_{Tok} vs. Ours _{Out}	61.11	68.05	71.43
Ours_{Tok} vs. Ours _{Act}	58.97	61.76	61.80

Table 5 Evaluation of intermediate-step rationality. We report the **win rate (%)** of AttriMem against each compared method, where higher values indicate that AttriMem is more frequently judged to produce more reasonable intermediate memory.

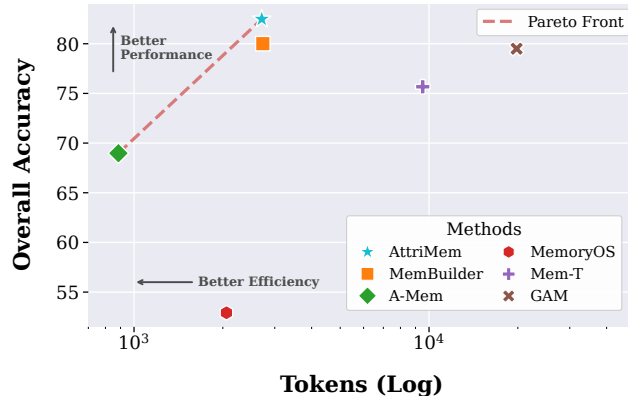


Figure 4 Performance–efficiency trade-off on LoCoMo. The horizontal axis denotes average inference token consumption per question, and the vertical axis denotes answer accuracy.

are independent, they can be evaluated in parallel within a single batch, adding roughly the latency of only one batched forward pass.

Dependence on attribution quality. AttriMem depends on accurate token-level attribution; thus, attribution errors may misassign credit, especially in long contexts. In our setting, this risk is limited because final answers usually rely on only a few retrieved records and their associated memory operations. Moreover, attribution rewards complement the global outcome reward and consistently improve the corresponding RL variants without observed regressions. Crucially, AttriMem is attribution-agnostic and can directly incorporate stronger long-context or white-box attribution methods.

Token-representable memory. Our current framework assumes that intermediate memory-module outputs are represented as textual records that can be decomposed into tokens and aligned with attribution scores. This makes the method naturally suitable for text-based memory systems, but direct extension to latent, vector-based, or graph-structured memory requires additional alignment mechanisms.

7 Conclusion

We presented AttriMem, an attribution-guided process-feedback framework for RL-based memory learning. AttriMem addresses the fine-grained credit-assignment bottleneck in existing memory-learning methods by tracing final-answer support back to intermediate memory contents and converting token-level contributions into local process rewards. Experiments on long-horizon dialogue question answering show that attribution-guided process feedback provides more precise and effective supervision than outcome-only or action-level rewards, improving overall performance, intermediate memory quality, and RL training stability.

References

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

- Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Mądry. 2024. Contextcite: Attributing model generation to context. *Advances in Neural Information Processing Systems*, 37:95764–95807.
- Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2026. [Agentic reinforced policy optimization](#). In *The Fourteenth International Conference on Learning Representations*.
- Yiming Du, Hongru Wang, Zhengyi Zhao, Bin Liang, Baojun Wang, Wanjun Zhong, Zezhong Wang, and Kam-Fai Wong. 2024. Perltqa: A personal long-term memory dataset for memory classification, retrieval, and fusion in question answering. In *Proceedings of the 10th SIGHAN Workshop on Chinese Language Processing (SIGHAN-10)*, pages 152–164.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, and 1 others. 2025. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25972–25981.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870.
- Wenbo Pan, Zhichao Liu, Xianlong Wang, Yu Haining, and Xiaohua Jia. 2026. [Towards long-horizon interpretability: Efficient and faithful multi-token attribution for reasoning LLMs](#). In *Forty-third International Conference on Machine Learning*.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Zhiyu Shen, Ziming Wu, Fuming Lai, Shaobing Lian, and Yanghui Rao. 2026. Membuilder: Reinforcing llms for long-term memory construction via attributed dense rewards. *arXiv preprint arXiv:2601.05488*.
- Yu Wang and Xi Chen. 2025. Mirix: Multi-agent memory system for llm-based agents. *arXiv preprint arXiv:2507.07957*.
- Yu Wang, Ryuichi Takanobu, Zhiqi Liang, Yuzhen Mao, Yuanzhe Hu, Julian McAuley, and Xiaojian Wu. 2025. Mem- $\{\alpha\}$: Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.
- Zhaofen Wu, Hanrong Zhang, Fulin Lin, Wujiang Xu, Xinran Xu, Yankai Chen, Henry Peng Zou, Shaowen Chen, Weizhi Zhang, Xue Liu, and 1 others. 2026. Gam: Hierarchical graph-based agentic memory for llm agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 34647–34664.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026a. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026b. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604.
- BY Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, and Zheng Liu. 2025. General agentic memory via deep research. *arXiv preprint arXiv:2511.18423*.

- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z Pan, and 1 others. 2026. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12805–12825.
- Yanwei Yue, Boci Peng, Xuanbo Fan, Jiaxin Guo, Qiankun Li, and Yan Zhang. 2026. Mem-t: Densifying rewards for long-horizon memory agents. *arXiv preprint arXiv:2601.23014*.
- Guilin Zhang, Wei Jiang, Xiejiashan Wang, Aisha Behr, Kai Zhao, Jeffrey Friedman, Xu Chu, and Amine Anoun. 2026. Adaptive memory admission control for llm agents. *arXiv preprint arXiv:2603.04549*.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731.
- Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. 2026. Are we ready for an agent-native memory system? *arXiv preprint arXiv:2606.24775*.
- Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*.
- Qiang Zhu and Jiajun Wu. 2026. Lapo: Leave-one-turn attribution for self-generated process rewards in multi-turn search reasoning. *arXiv preprint arXiv:2607.13501*.

Symbol	Meaning
a_t^{mem}	Memory-construction action sampled at session t .
$h_t \in \mathcal{H}$	Session t in the conversation history.
τ^{mem}	Memory-construction trajectory.
$\mathcal{M}_t$	Memory store after processing session t .
$\Delta\mathcal{M}_t$	Textual memory content produced or modified by a_t^{mem} .
π_θ	Agent policy parameterized by θ .
G	GRPO group size.
$q \in \mathcal{Q}$	Query from query set.
$r(\cdot)$	Reward function.
$\phi(z_i, y; c)$	Attributed contribution of source z_i on response y under full context c .

Table 6 Summary of notations.

Appendix

A ContextCite Methodology Details

We instantiate the attribution function ϕ with ContextCite (Cohen-Wang et al., 2024), adapting its source-level formulation to the token-level memory outputs used by AttriMem. ContextCite permits a source to be any subset of context tokens and also studies fine-grained word-level sources; here, we use individual model tokens as sources. For a generated answer $\hat{y}$, let the attribution context be $c = (z_1, \dots, z_d)$, where each source z_j is a token from the textual memory content and retains its provenance to the memory action that produced or modified it. The objective is to estimate how retaining each source changes the answer model’s score for the fixed answer $\hat{y}$.

Context ablations. We represent an ablation by a binary inclusion vector $v \in \{0, 1\}^d$, where $v_j = 1$ indicates that source z_j is retained and $v_j = 0$ indicates that it is removed. Let $\text{Ablate}(c, v)$ denote the context formed by retaining the selected sources in their original order. Following ContextCite, we independently retain each source with probability 1/2, which is equivalent to sampling v uniformly from $\{0, 1\}^d$. For N sampled masks $\{v_k\}_{k=1}^N$, the corresponding ablated contexts can be evaluated in one batch.

Scoring the fixed answer. Let p_{ans} denote the fixed answer model used to score $\hat{y} = (\hat{y}_1, \dots, \hat{y}_L)$. For each ablation vector, we evaluate the probability of the same generated answer under teacher forcing:

$$\begin{aligned} f(v_k) &:= p_{\text{ans}}(\hat{y} \mid q, \text{Ablate}(c, v_k)) \\ &= \prod_{\ell=1}^L p_{\text{ans}}(\hat{y}_\ell \mid q, \text{Ablate}(c, v_k), \hat{y}_{<\ell}). \end{aligned} \quad (8)$$

As in ContextCite, we regress on the logit-transformed probability

$$g(v_k) := \sigma^{-1}(f(v_k)) = \log \frac{f(v_k)}{1 - f(v_k)}, \quad (9)$$

rather than on the bounded probability itself. Holding $\hat{y}$ fixed across ablations isolates how the included memory sources affect the model’s preference for the observed answer; it does not resample an answer for each ablated context.

Sparse linear surrogate. ContextCite approximates $g(v)$ with a linear surrogate $\hat{g}(v) = b + w^\top v$. Given the ablation dataset $\{(v_k, g(v_k))\}_{k=1}^N$, we fit the surrogate with LASSO:

$$(\hat{b}, \hat{w}) = \arg \min_{b, w} \frac{1}{2N} \sum_{k=1}^N (g(v_k) - b - w^\top v_k)^2 + \alpha \|w\|_1. \quad (10)$$

The sparsity penalty reflects the assumption that only a small subset of the context materially affects a particular answer and allows the surrogate to be estimated from substantially fewer masks than sources.

Because the surrogate is linear in the inclusion indicators, its fitted coefficient for source z_j is used directly as the attribution score:

$$\phi(z_j, \hat{y}; c) := \hat{w}_j. \quad (11)$$

A positive coefficient means that retaining the source increases the score of $\hat{y}$ under the local surrogate, whereas a negative coefficient indicates that retaining it decreases the score.

Mapping attribution to process rewards. Each source index is mapped back to the memory update from which it originated. For update $\Delta\mathcal{M}_t$, let $I_t \subseteq \{1, \dots, d\}$ denote its attributed token indices and let $l_t = |I_t|$. We aggregate their ContextCite coefficients to obtain the process reward

$$r_t = \begin{cases} \frac{1}{\sqrt{l_t}} \sum_{j \in I_t} \hat{w}_j, & l_t > 0, \\ 0, & l_t = 0, \end{cases} \quad (12)$$

which is the reward used for memory action a_t^{mem} in §4.2. The square-root normalization reduces the tendency of longer memory updates to receive larger-magnitude rewards solely because they contain more tokens.

Ablation budget. The original ContextCite experiments use 32 independently sampled ablations and LASSO regularization $\alpha = 0.01$. We use the same maximum ablation budget and increase the cumulative number of masks from 8 to 16 and then 32 at RL steps 0, 80, and 200, respectively. This schedule reduces attribution cost early in training while improving the fidelity of process rewards as the memory policy becomes stronger.

B Implementation Details

Memory policy and fixed QA interface. We use Qwen3-4B as the memory-construction policy for all AttriMem variants. At session t , the policy receives the current session h_t and the previous memory store $\mathcal{M}_{t-1}$ and produces a memory action a_t^{mem} . The memory store contains core, episodic, semantic, and procedural modules; the same policy backbone is used with action-specific prompts and action spaces. After all sessions have been processed, a frozen retriever selects information from $\mathcal{M}_T$ and a separate answer model generates $\hat{y}$. The retriever and answer model are used to obtain training feedback but are not optimized as part of the memory policy.

Two-stage training. We train the memory policy on LongMemEval in two stages. The SFT stage provides the initialization used by the SFT and SFT+RL variants and runs for 10 epochs (3,000 update steps), with learning rate 5×10^{-7} and batch size 32. Starting from either the base model or the SFT initialization, the RL stage applies GRPO for 400 update steps with learning rate 1×10^{-6} , effective batch size 256, group size $G = 8$, and maximum sequence length 6,000. The GRPO objective includes KL regularization against the reference policy. For each training example, the old policy samples G memory-construction trajectories; the fixed retrieval-and-answering interface evaluates each resulting memory store, and the answer rewards are normalized within the group. The per-action advantage combines the resulting trajectory-level outcome advantage with the attribution-based process advantage defined in §4.2.

Attribution computation. We provide the complete ContextCite formulation and its token-level adaptation in Appendix A. For each generated answer, we keep the answer fixed while evaluating randomly ablated memory contexts, fit the sparse linear surrogate, and map each token coefficient back to the memory action that produced the token. The cumulative ablation budget increases from 8 to 16 and then 32 masks at RL steps 0, 80, and 200, respectively. All masks for an attribution estimate are evaluated in parallel as a batch.

Controlled comparison and evaluation. Following MemBuilder (Shen et al., 2026), we use the same Qwen3-4B backbone, memory architecture, SFT initialization, optimization budget, shared hyperparameters, retrieval-and-answering interface, decoding configuration, and evaluation protocol for the controlled comparison. The designed difference is the RL reward: AttriMem supplements the outcome reward with attribution-based token-level process rewards. We train on LongMemEval and evaluate zero-shot transfer to LoCoMo and PerLTQA, reporting accuracy on all three benchmarks. The answer-model comparison additionally evaluates the learned memory with Claude 4.5 Sonnet, GPT-4.1, and Qwen3-4B readers, as reported in Table 3.

Hyperparameter	Value
Backbone	Qwen3-4B
SFT learning rate	5e-7
RL learning rate	1e-6
SFT batch size	32
RL effective batch size	256
GRPO group size	8
Training steps (SFT)	3000
Training steps (RL)	400
Max sequence length	6000

Table 7 Training hyperparameters for AttriMem.

LLM judge protocol (for §5.4). The LLM judge receives the following context for each pairwise comparison: (1) the full original dialogue, (2) the question, (3) the gold answer, (4) the intermediate outputs of System A and System B at the specified stage (extraction / compression / retrieval). Method identities are replaced with “System A” and “System B”. Each pair is evaluated twice (order A-then-B and B-then-A); the final win rate averages both orderings. The actual prompt is shown in Figure 5.

Hyperparameters. Table 7 lists all training hyperparameters.

Compute. We use 4× NVIDIA H800 (80GB) GPUs. For AttriMem, SFT training time is ∼30 hours while RL training time is ∼5 days.

Reproducibility. Code and trained model checkpoints will be released upon acceptance. The LongMemEval, LoCoMo, and PerLTQA datasets are all publicly available; download instructions are included in the repository.

C Extended Related Work

Process rewards for agents via attribution. A significant challenge in long-horizon reasoning or agentic tasks is reward sparsity, since outcome rewards are more convenient to obtain than process rewards. Process rewards are important for removing redundancy, improving problem-solving efficiency and enforcing critical intermediate milestones. There are several common sources of process rewards and attribution is a mainstream direction, where we estimate the contribution of intermediate steps to final outcomes. Tree-branch reward approaches, represented by *Agentic Reinforced Policy Optimization (ARPO)* (Dong et al., 2026), assign rewards to intermediate steps based on how likely the downstream branches from a step are likely to lead to correct outcomes.

Our work is among the first to adopt a context-attribution-based approach to process rewards. *Leave-One-Turn Attribution for Policy Optimization (LOTAPO)* (Zhu and Wu, 2026) is especially relevant to our work and it estimates the contribution of a turn to final golden answer. One main distinction between LOTAPO and AttriMem is that, LOTAPO corresponds to *action-level* attribution-derived process rewards while we obtain *token-level* rewards at a finer granularity. We also show in Table 4 that token-level rewards is superior to action-level rewards.

Attribution to reasoning steps. Pan et al. (2026) proposed recursive attribution to enable attribution to long reasoning traces in reasoning LLMs for faithful and efficient interpretability.

Prompt template for evaluating intermediate memory-construction quality.

You are evaluating intermediate memory-construction quality for a long-term memory QA system.

Given:

1. A conversation session.
2. Downstream QA targets that the memory system should eventually help answer.
3. Two systems' intermediate memory construction outputs for the same session. The outputs may contain one memory module or all memory modules together.

Your job is to decide which system produced the more evidence-grounded, useful, and correct memory construction output for this step.

Use this priority order:

1. Evidence-grounded faithfulness: prefer memory updates that are directly traceable to concrete evidence in the conversation session. Penalize hallucinated facts, wrong dates, wrong speakers, unsupported causal claims, and plausible but ungrounded details.
2. Answer-critical coverage: prefer memories that preserve facts needed to answer the downstream questions, including entities, dates, counts, relationships, preferences, events, objects, motivations, and reasons.
3. Operation rationality: ADD, UPDATE, MERGE, APPEND, REPLACE, REWRITE, and SKIP should be appropriate given the relevant existing memories. Penalize duplicate ADDs, unnecessary updates, overwriting useful facts, and failing to update changed facts.
4. Module fit: judge whether the output belongs in this memory module.
 - core: stable user identity, relationships, preferences, values, long-term goals, recurring traits.
 - episodic: timestamped events and experiences.
 - semantic: stable facts about people, places, objects, and concepts in the user's world.
 - procedural: reusable procedures, workflows, recipes, or instructions. If there is no procedural content, SKIP or an empty operation can be best.
5. Specificity and retrievability: concise but concrete memories with good anchors are better than vague summaries.
6. Non-redundancy: avoid duplicate, bloated, or low-value memories.

Important rules:

- The gold answers are hints about important facts, not permission to add facts unsupported by this session.
- First identify which QA targets are actually supported by this conversation session; ignore unrelated targets when choosing a winner.
- In each module output, `existing_memory_count` and `relevant_existing_memories` are context only; judge the current operations and newly written/updated memory content.
- Do not prefer longer output or more operations by default.
- Do not reward style unless it improves retrieval or factual clarity.
- If this session is unrelated to the listed QA targets, judge long-term memory usefulness for future QA.
- If both are similarly good or similarly bad, choose Tie.
- Use the scoring fields to make the comparison explicit, but the final winner should follow the priority order above.

Conversation session:

{session}

Most relevant downstream QA targets:

{questions}

Gold answer(s):

{gold_answers}

Memory module scope:

{module_scope}

System A output:

{system_a_steps}

System B output:

{system_b_steps}

Return JSON only:

```
{{
  "scores": {{
    "evidence_grounding_faithfulness": {{ "A": 1, "B": 1}},
    "answer_critical_coverage": {{ "A": 1, "B": 1}},
    "operation_rationality": {{ "A": 1, "B": 1}},
    "module_fit": {{ "A": 1, "B": 1}},
    "retrievability": {{ "A": 1, "B": 1}},
    "non_redundancy": {{ "A": 1, "B": 1}}
  }},
  "winner": "A" | "B" | "Tie",
  "reason": "one concise sentence"
}}
```

Figure 5 Prompt template for evaluating the quality of intermediate memory-construction outputs in a long-term memory QA system.